

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ «ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Отчет по алгоритмам и структурам данных

Тема: Анализ и оптимизация алгоритма поиска максимальной суммы
подотрезка

Студент: Ошаров Александр

Семинарист: Маров Руслан

1. Постановка задачи

Необходимо исследовать алгоритм поиска максимальной суммы подотрезка фиксированной длины k в массиве чисел. Требуется определить асимптотическую верхнюю границу сложности исходного алгоритма, предложить его оптимизацию, разработать оптимизированный алгоритм и обосновать его сложность.

2. Анализ исходного алгоритма

Исходный алгоритм реализован в виде функции на языке C++:

```
int long_find_max_sum(const std::vector<int>& arr, int  
k) {  
    int n = arr.size();  
    int max_sum = INT_MIN;  
    for (int i = 0; i <= n - k; ++i) {  
        int current_sum = 0;  
        for (int j = i; j < i + k; ++j) {  
            current_sum += arr[j];  
        }  
        max_sum = std::max(max_sum, current_sum);  
    }  
    return max_sum;  
}
```

Алгоритм перебирает все возможные подотрезки длины k , каждый раз заново вычисляя их сумму. Внешний цикл выполняется $O(n)$ раз, а внутренний цикл — $O(k)$ раз, что приводит к итоговой временной сложности $O(nk)$. При $k \approx n$ получаем верхнюю границу сложности $O(n^2)$. Использование памяти — $O(1)$, так как дополнительно хранится лишь несколько переменных.

3. Оптимизация алгоритма

Оптимизация заключается в применении техники «скользящего окна». Вместо пересчета суммы каждого подотрезка заново, можно на каждом

шаге обновлять сумму за $O(1)$, вычитая первый элемент предыдущего окна и добавляя новый элемент.

```
int optimized_find_max_sum(const std::vector<int>& arr,  
int k) {  
    int n = arr.size();  
    if (k > n || k <= 0) {  
        throw std::invalid_argument("Invalid window size");  
    }  
  
    int current_sum = 0;  
    for (int i = 0; i < k; ++i) {  
        current_sum += arr[i];  
    }  
  
    int max_sum = current_sum;  
  
    for (int i = k; i < n; ++i) {  
        current_sum += arr[i] - arr[i - k];  
        max_sum = std::max(max_sum, current_sum);  
    }  
  
    return max_sum;  
}
```

Оптимизированный алгоритм выполняет инициализацию суммы первых k элементов за $O(k)$, а далее обновляет сумму для каждого из оставшихся подотрезков за $O(1)$. Итоговая временная сложность составляет $O(n)$, использование памяти — $O(1)$.

4. Выводы

Анализ показал, что исходный алгоритм имеет асимптотическую сложность $O(nk)$, что в худшем случае приводит к квадратичной зависимости. Оптимизированный алгоритм на основе техники скользящего окна позволяет снизить сложность до линейной $O(n)$,

сохранив использование памяти на уровне $O(1)$. Таким образом, предложенный метод является эффективным и предпочтительным для практического применения.

5. Список литературы

1. Кормен Т. и др. Алгоритмы: построение и анализ. — М.: Вильямс, 2005.
2. Седжвик Р., Уэйн К. Алгоритмы на C++. — М.: Вильямс, 2015.
3. ГОСТ 7.32–2017. Отчет о научно-исследовательской работе. Структура и правила оформления.